Google Reichman University | AI Tech School

Hash Set & Map

Izik Zur

Hash Set Definition

- Definition:
  - Sequence of elements
  - Key, unique id of any item, used to Store/retrieve Item from Container
- Data Structure- optimize for Elements Insert/Search/Remove.
- Hash- O(1)- small amount of additional memory consumption
- All entries in the Set Container is associated to at most one entry (no duplication)

Hash Set

- **Simple Algorithm:**
  - Use Hash Function to find empty Item entry in set
  - In Case of Collision – re-hash
- **On Search: if not found in first hash try to re-hash**
  - Until found , Empty , or max re-hash
- **On Remove – signal Entry is empty for insertion but value is invalid (not end of search)**
- **Collision Options**
  - In General ReHash: nextEntry= Hash(f(key));.
  - ReHash = hash (key+ PreviousHashresult)
  - ReHash = hash (key+ X)

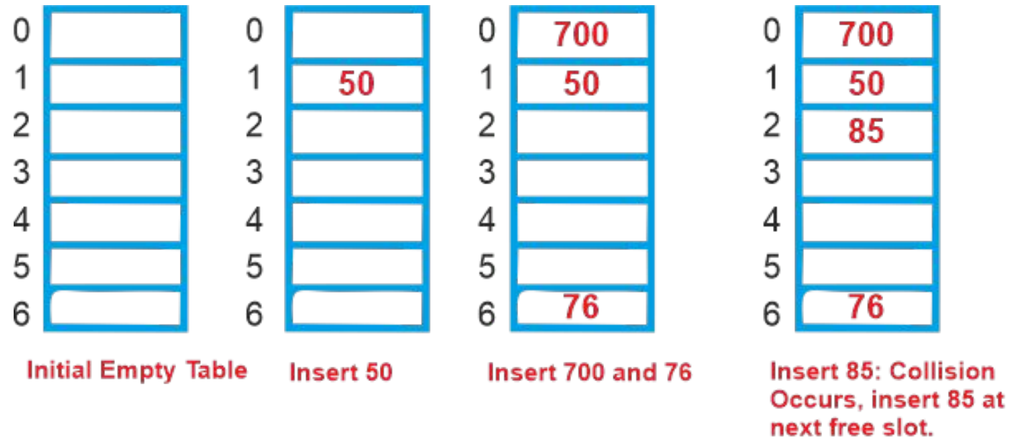
  - Use Google to find Hash functions and re-Hash

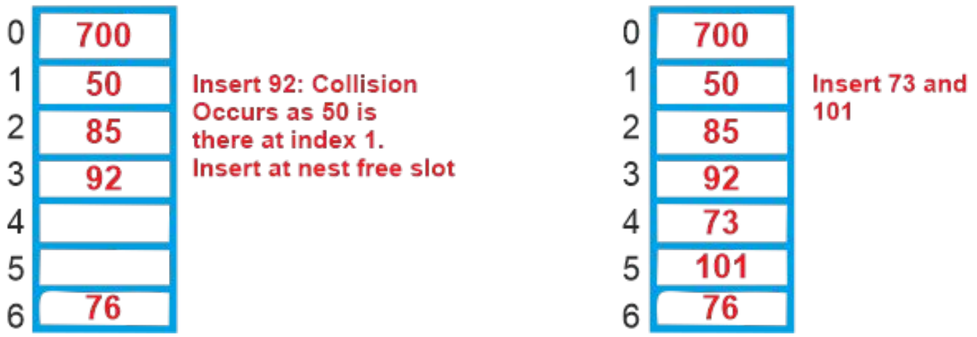
Hash Set Collision & ReHash

If slot hash (x) % Size is full, then we try
(hash (x) + 1) % Size
If (hash (x) + 1) % Size is also full, then we try
(hash (x) + 2) % Size
If (hash (x) + 2) % Size is also full, then we try
(hash (x) + 3) % Size

Hash Set Collision

Let's use "key mod 7" as a simple hash function with the following keys: 50, 700, 76, 85, 92, 73, 101





Optimize Collisions and ReHashes

- Increase the table size vs the capacity by factor (30%)
- Change the size to be a prime number (Why?)
- Choose good hash function so that the keys will be well distributed in the table

Hash Map

- Definition:
  - Sequence of elements
  - Pair of Key - value,
- Data Structure- optimize for Elements Insert/Search/Remove.
- Hash- O(1)- small amount of additional memory consumption
- The key should be unique. The value shouldn’t be unique, it also could be NULL

Hash Map

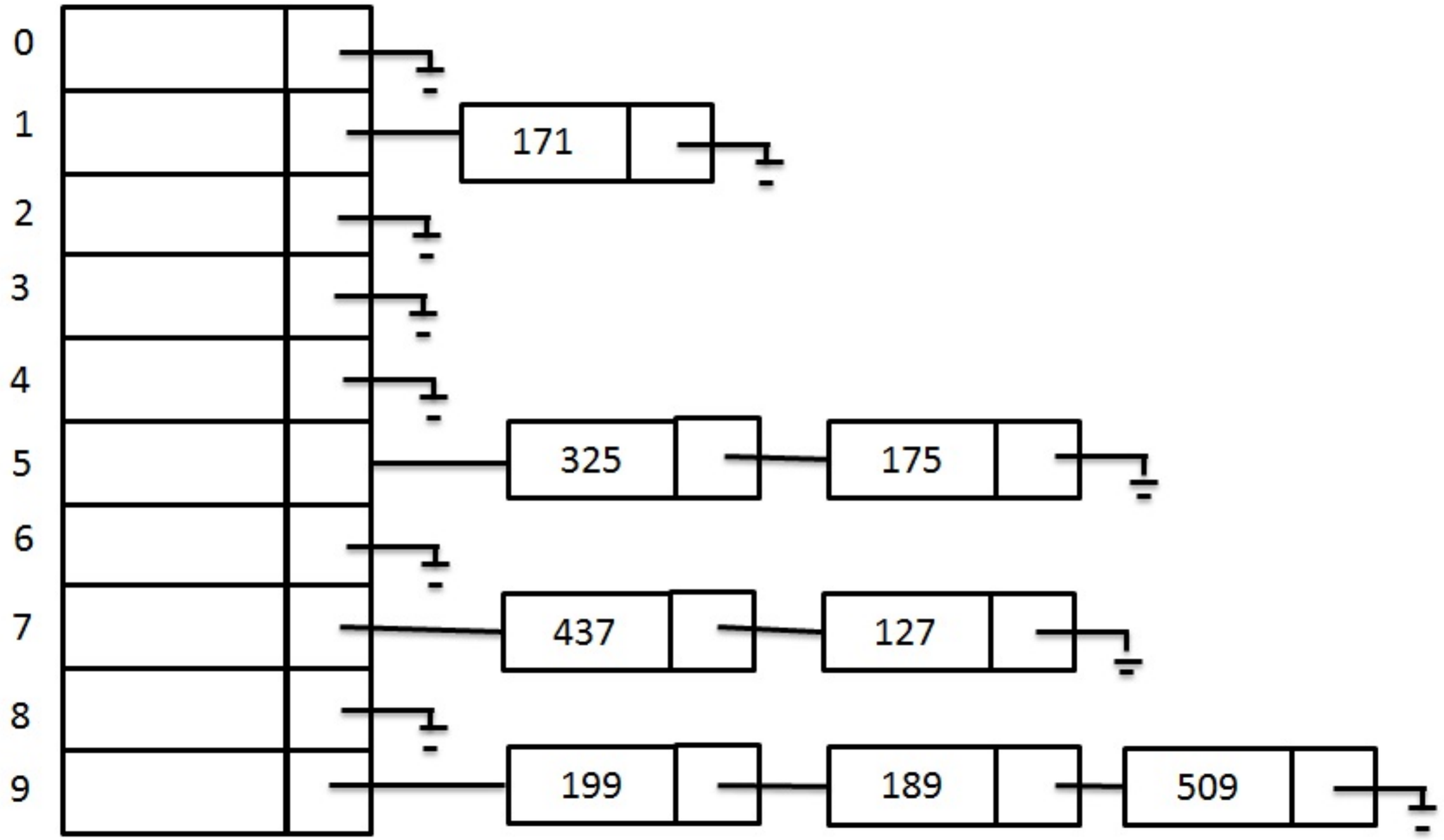
- Simple Algorithm:

  - Use Hash Function to find the index in the table that it should inside the backet

- On Search: **On Remove** - Use Hash Function to find the index in the table that need to find the key in the backet. If found – return True for search or remove the key-value in case of remove.

  - Use Google to find Hash functions and re-Hash

Hash Map Implementation



Optimize Collisions and ReHashes

- Change the size to be a prime number (Why?)
- Choose good hash function so that the keys will be well distributed in the table

Hash Map structure

```
struct HashMap
{
List**    m_data;
size_t (*m_hashFunction)(void* _data));
int (*m_equalityFunction)(void* _firstData, void* _secondData);
size_t  m_hashSize; /*original size given by the client*/
size_t m_capacity /*real hash size */
size_t  m_numOfItems; /*number of occupied places in the table*/
};
```

Hash-Map API's

```
typedef struct HashMap HashMap;
typedef enum Map_Result
{
MAP_SUCCESS = 0,
MAP_UNINITIALIZED_ERROR,      /**< Uninitialized map error     */
MAP_KEY_NULL_ERROR,           /**< Key was null       */
MAP_KEY_DUPLICATE_ERROR,      /**< Duplicate key error      */
MAP_KEY_NOT_FOUND_ERROR,      /**< Key not found     */
MAP_ALLOCATION_ERROR          /**< Allocation error    */
} Map_Result;
typedef size_t (*HashFunction)(void* _key);
typedef int (*EqualityFunction)(void* _firstKey, void* _secondKey);
typedef int (*KeyValueActionFunction)(const void* _key, void* _value, void* _context);
```

Hash-Map API's-2

HashMap* HashMap_Create(size_t _capacity, HashFunction _hashFunc, EqualityFunction _keysEqualFunc);

void HashMap_Destroy(HashMap** _map, void (*_keyDestroy)(void* _key), void (*_valDestroy)(void* _value));

Map_Result HashMap_Rehash(HashMap *_map, size_t newCapacity);

Map_Result HashMap_Insert(HashMap* _map, const void* _key, const void* _value);

Map_Result HashMap_Remove(HashMap* _map, const void* _searchKey, void** _pKey, void** _pValue);

Map_Result HashMap_Find(const HashMap* _map, const void* _key, void** _pValue);

size_t HashMap_Size(const HashMap* _map);

size_t HashMap_ForEach(const HashMap* _map, KeyValueActionFunction _action, void* _context);

Exercise

Implement ADT Hash-Map,

THANK YOU

GOOD LUCK